



DIPLOMADO

Desarrollo de sistemas con tecnología Java

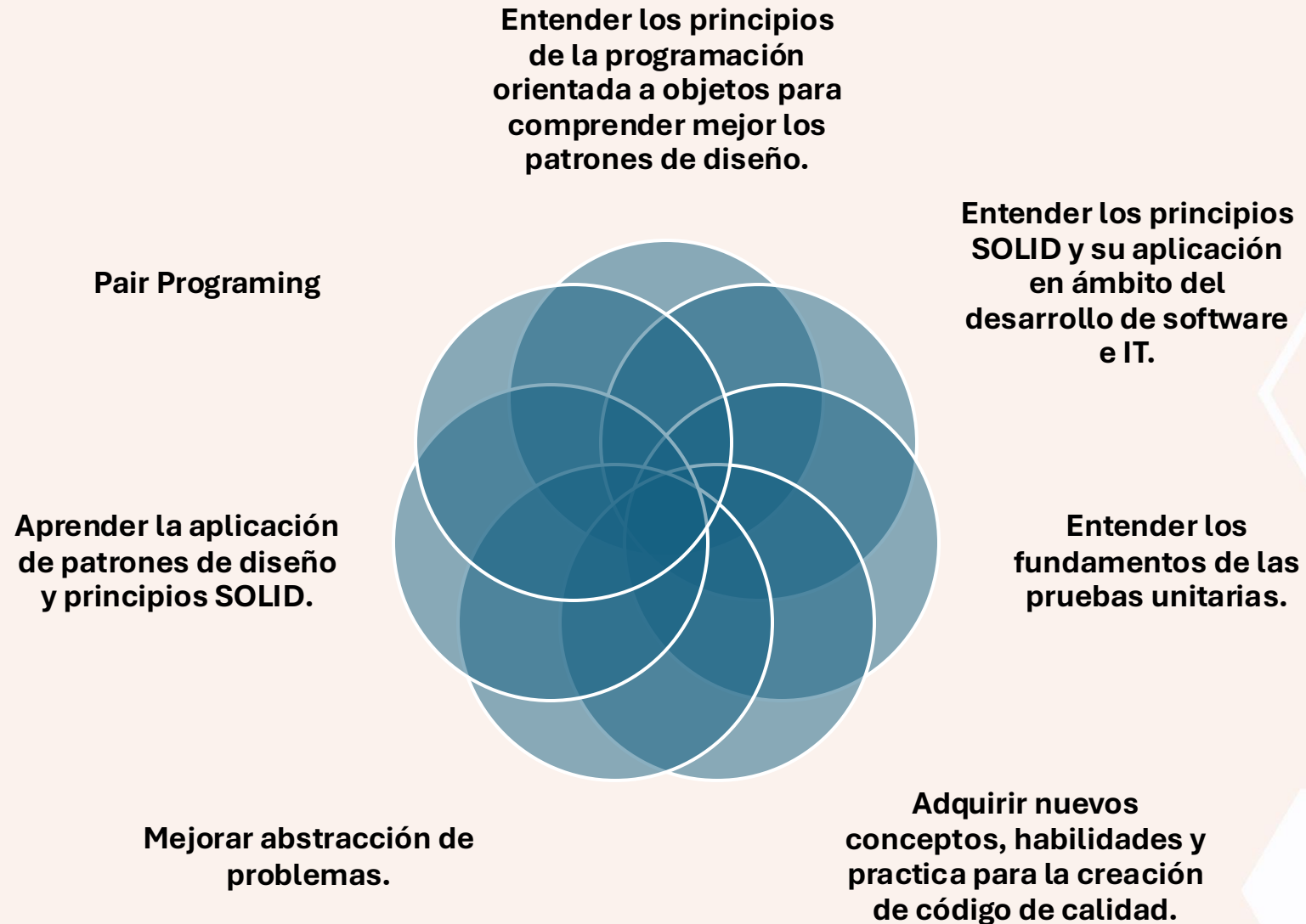
Módulo 3

Principios y patrones de diseño

Ing. Alfonso Hernández Xochipa



Objetivos del módulo



¿Quien es su instructor?

- Experiencia profesional
- ¿A que me dedico?
- Certificaciones y estudios
- Hobbies
- ¿Por que elegí aportar al diplomado?



10%**Prácticas / Ejercicios Iniciales**

- Diseño de diagrama de clases UML a partir de código.
- Desarrollo de código a partir de un diagrama UML
- Ejercicios en Clase (Pair programming)

30%**Principios SOLID (5)**

- Ejercicios sobre la implementación de los 5 principios solid

30%**Patrones de diseño**

- Ejercicios de implementación de patrones de diseño.

30%**Avance Proyecto Final**

- Regresión y refactorización de diagrama relacional realizado en el modulo 2.

Evaluación

100%

Consideraciones

- Hacer uso del icono de 🖐 para pedir la palabra.
- Si por alguna circunstancia entras tarde a la sesión, no activar el micrófono para evitar interrumpir la sesión.
- Se prohíbe la violencia, agresión y el uso de palabras indebidas contra cualquiera de los participantes de la sesión.

Acuerdos

- Recesos 30 minutos.
 - Después de las primeras 2 horas

Adicionales

El área de IT es un universo demasiado extenso con muchas galaxias que explorar.

Como parte del aprendizaje, es necesario que adquieras otro tipo de skills, cultura general, cultura tecnológica y puedas ir descubriendo esas galaxias.

Cine o TV



Lectura



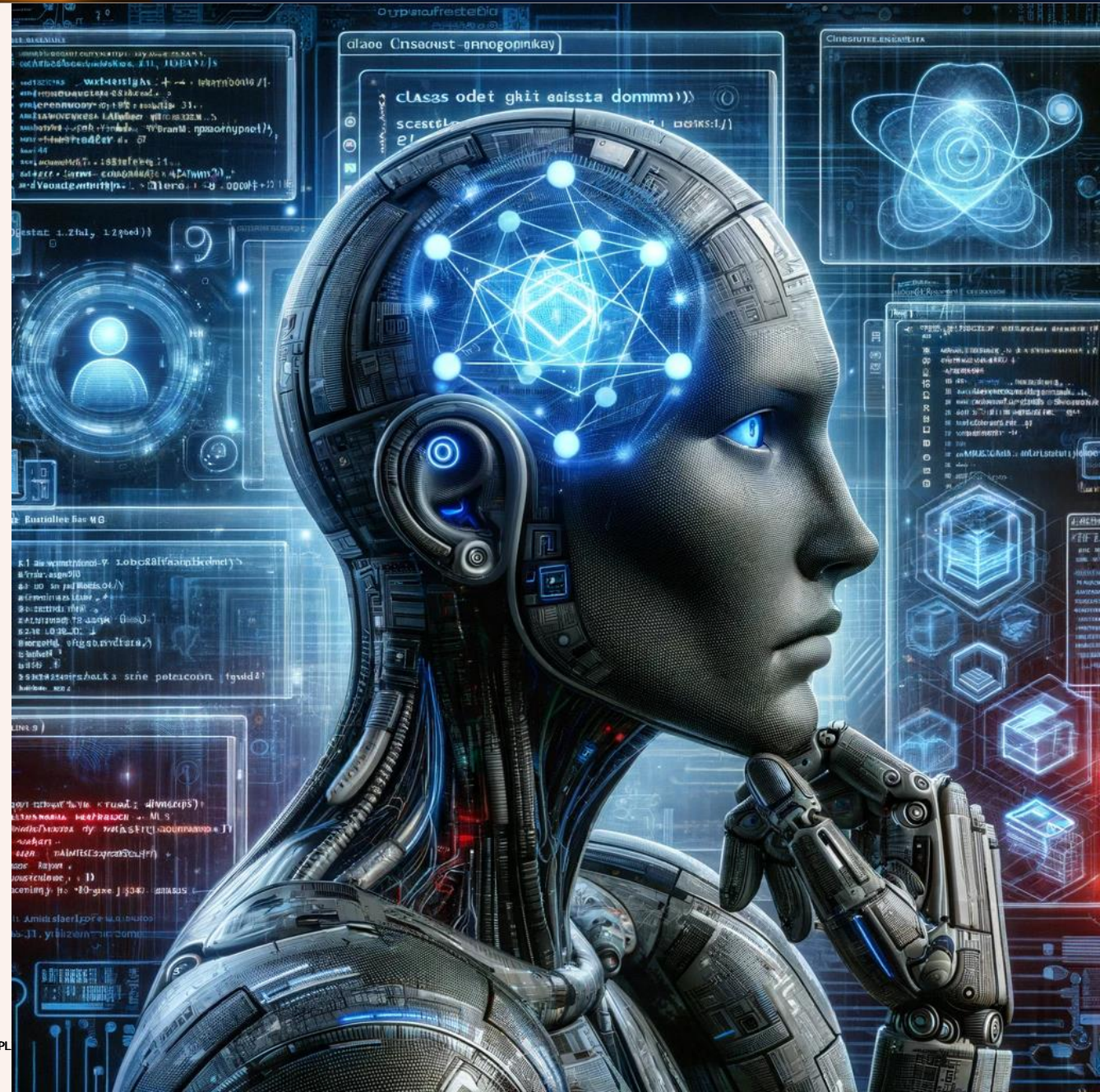
¿Que veremos hoy?

Terminología
POO(Refresh).

Principios de
diseño orientado
a objetos.

Diagramas de
clases y
relaciones entre
clases con UML.

Principios de
pruebas
unitarias.



Terminología de programación orientada a objetos

Clase

Una **Clase** es una estructura tipo plantilla de donde se pueden crear **Objetos**. Una clase tiene atributos, métodos y propiedades de visibilidad.

```
package com.dgtic.dstj15.poobasicos.clase;  
public class Computadora {  
  
    String version;  
    String procesador;  
    int ram;  
  
    public Computadora(){  
  
    }  
  
    public void encender(){  
  
    }  
    public void apagar(){  
  
    }  
    public void operar(){  
  
    }  
}
```

Terminología de programación orientada a objetos

Instancia

Para crear una nueva **instancia** de una clase se utiliza la palabra reservada **new** lo cual permite crear nuevos **objetos**.

Un objeto que sea una **instancia(ejemplar)** de clase, tendrá valores predeterminados para sus atributos, a esto se le llama **estado actual** del objeto.

```
public class Cluster {  
    Computadora cc01;  
    private Computadora cc02;  
    private List<Computadora> cst01;  
  
    public Cluster(){  
        cc01=new Computadora();  
        cc02=new Computadora();  
        cst01.add(cc01);  
        cst01.add(cc02);  
    }  
}
```

Terminología de programación orientada a objetos

Modificadores de acceso

Cuando se declaran variables de instancia o variables de otra índole podemos manejar su **visibilidad** mediante **modificadores de acceso**.

- **Private**
- **Public**
- **Default (Package-private)**
- **Protected**

```
public class Computadora {  
  
    public String version;  
    protected String procesador;  
    protected int ram;  
    private String uuid;  
  
    public Computadora(){  
  
    }  
  
    public void encender(){  
  
    }  
    public void apagar(){  
  
    }  
    public void operar(){  
  
    }  
  
}
```

Terminología de programación orientada a objetos

Encapsulación

La **encapsulación** se lograra cuando una clase externa nunca pueda acceder a los atributos y métodos privados de otra clase. Solo podrán tener acceso los métodos que pertenezcan a la misma clase.

```
public class Computadora {  
  
    String version;  
    public String procesador;  
    public int ram;  
    String uuid;  
  
    public Computadora(){  
    }  
}
```



```
public class Computadora {  
  
    private String version;  
    private String procesador;  
    private int ram;  
    private String uuid;  
  
    public Computadora(){  
    }  
  
    public String obtenerVersion(){  
        return version;  
    }  
    public String obtenerProcesador(){  
        return version;  
    }  
    public int obtenerRam(){  
        return ram;  
    }  
}
```



Contexto de variables

Una diferencia entre **variable de clase** y **variable local** es que una **variable de clase** puede ser accedida a cualquier método que se encuentre dentro de la clase de donde se creó, sin embargo una **variable local** solo se encuentra disponible en un contexto específico.

```
public class Computadora {  
  
    public String version;  
    protected String procesador;  
    protected int ram;  
    private String uuid;  
  
    public Computadora(){  
  
    }  
  
    public String obtenerVersion(){  
        return version;  
    }  
    public String obtenerProcesador(){  
        return version;  
    }  
    public int obtenerRam(){  
        return ram;  
    }  
}
```

Herencia

La **herencia** de clases permite la **reutilización** de código de una manera eficiente. Cuando una clase hereda de otra se heredan atributos, métodos y clases internas que se encuentren permitidas mediante el modificador de acceso.

- **Herencia simple.**
- **Extends**
- **Super**

```
public class ComputadoraPersonal extends Computadora{  
  
    public ComputadoraPersonal(){  
        super();  
        super.procesador="IntelI9";  
        super.ram=32;  
        super.version="vxx012";  
    }  
  
    @Override  
    public void apagar() {  
        super.apagar();  
    }  
  
    ...  
}
```

Terminología de programación orientada a objetos

Polimorfismo

El **polimorfismo** es la capacidad de un objeto de tomar muchas formas diferentes.

El polimorfismo permite la **reutilización de código**, **código generico**, **extensibilidad**, **mantenimiento flexible**.

```
1 String name = "Jhon Doe";
2 Integer years = 24
3 Double heigth = 180.02
4 Double width = 100.2
5
6 String description = "My name is"+name; // My name is Jhon Doe
7 Integer nextYears = years +1; // 25
8
```

Ejemplo: el símbolo “+” dentro de la programación puede tomar muchas formas diferentes de su uso.

```
public class Cluster {
    private Computadora cc01;
    private Computadora cc02;

    public Cluster(){
        // Polimorfismo a su maxima expresion
        this.cc01=new ComputadoraPersonal();
        this.cc02=new Mac();
    }

    public Computadora getFirstComputer(){
        return cc01;
    }
    public Computadora getSecondComputer(){
        return cc02;
    }
}
```

*Las **clases abstractas** e **interfaces** son herramientas importantes para lograr que el polimorfismo se implemente en Java.*

Interfaz

Colección de métodos abstractos(sin implementación) que una clase debe implementar si quiere cumplir con esa interfaz.

Hablemos de un **contrato** que la clase implementadora debe de cumplir.

Para implementar una **interfaz** se utiliza la palabra reservada ***implements***.

```
public interface ClusterService {  
    public Computadora getFirstComputer();  
    public Computadora getSecondComputer();  
    public boolean isClusterActive();  
}  
  
public class Cluster implements ClusterService{  
    private Computadora cc01;  
    private Computadora cc02;  
    private boolean active;  
  
    public Cluster(){  
  
        this.cc01=new ComputadoraPersonal();  
        this.cc02=new Mac();  
    }  
  
    @Override  
    public Computadora getFirstComputer() {  
        return cc01;  
    }  
  
    @Override  
    public Computadora getSecondComputer() {  
        return cc02;  
    }  
  
    @Override  
    public boolean isClusterActive() {  
        return active && cc01!=null;  
    }  
}
```


Terminología de programación orientada a objetos

Clase Abstracta

Una **clase abstracta** es una clase que **no puede ser instanciada**, solo puede ser utilizada para crear **subclase** mediante la **herencia**.

Puede contener **métodos abstractos** y **métodos con implementación**.

```
public abstract class ClusterBase{
    protected List<Computadora> nodos;
    protected boolean estaActivo;
    protected Date fechaReinicio;

    protected ClusterBase(){
        this.nodos=new ArrayList<>();
        this.estaActivo=false;
        this.fechaReinicio=new Date();
    }

    protected void activarCluster(){
        this.estaActivo=true;
    }

    protected void reiniciar(){
        estaActivo=false;
        fechaReinicio=new Date();
        estaActivo=true;
    }

    abstract void agregarNodos();
    abstract void limpiarNodos();
}
```

Clase inmutable

Una **Clase Inmutable** es aquella que solo permite la lectura de sus atributos. Establece una maxima encapsulacion e integridad de sus datos.

- No apta para herencia
- Seguridad en Hilos
- Uso en Cache
- Programacion Funcional

```
public final class Computadora {  
  
    private String version;  
    private String procesador;  
    private int ram;  
    private String uuid;  
  
    public Computadora(){  
  
    }  
  
    public String obtenerVersion(){  
        return version;  
    }  
    public int obtenerRam(){  
        return ram;  
    }  
  
    public String obtenerUuid(){  
        return uuid;  
    }  
  
    public String obtenerProcesador(){  
        return procesador;  
    }  
}
```

Static Context

- El contexto “static” es un bloque de código que se almacena en la JVM y se inicializa antes que todas las instancias y clases.
- Se almacena en la memoria Heap.
- Se pueden llamar variables, métodos o clases internas sin necesidad de instancia.

```
1 public class ComputerUtils{
2     public static final int RAM_DEFAULT=16;
3     public static final int SSD_DEFAULT=256;
4     public static final int UUID="abc-dbe-123X12"
5
6     public static String getDefaultComputerLabel(final String model){
7         return "This is a "+model+" computer with "+
8             ComputerUtils.RAM_DEFAULT+" RAM and "+
9             SSD_DEFAULT+" SSD by default.";
10    }
11
12    static class ComputerBuilder{
13        static Computer build(final int ram, final int ssd, String name){
14            return new Computer(ram,ssd,name);
15        }
16    }
17 }
```

Bonus 🌟 Tipo Record

Tipo de dato especial
introducido en Java 16.
Permite definir clases
inmutables de manera mas
eficiente y concisa.

- Inmutabilidad
- Metodos automaticos
- Getters implicitos
- No herencia
- Solo datos

```
public record Computadora(  
    String version,  
    String procesador,  
    int ram,  
    String uuid) {  
  
    public Computadora{ // Constructor  
        // Validate something  
    }  
}
```

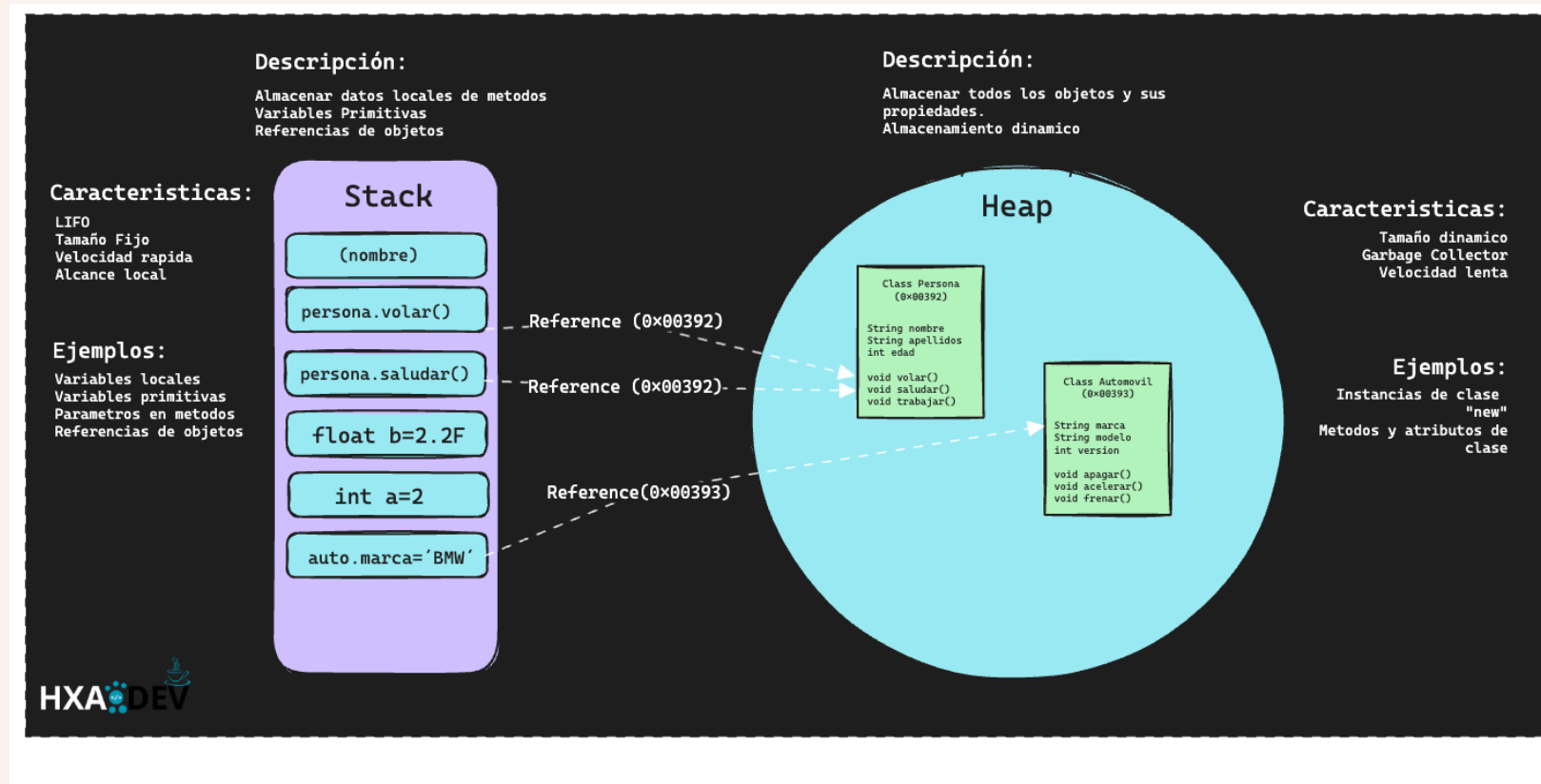
Practica 1.1



Terminología de programación orientada a objetos

Stack - Heap

Cada **objeto** o **instancia** tiene un espacio reservado en **memoria**.



Hablemos de objetos

Comportamiento

Que se puede hacer con el objeto

Estado

Como reacciona el objeto

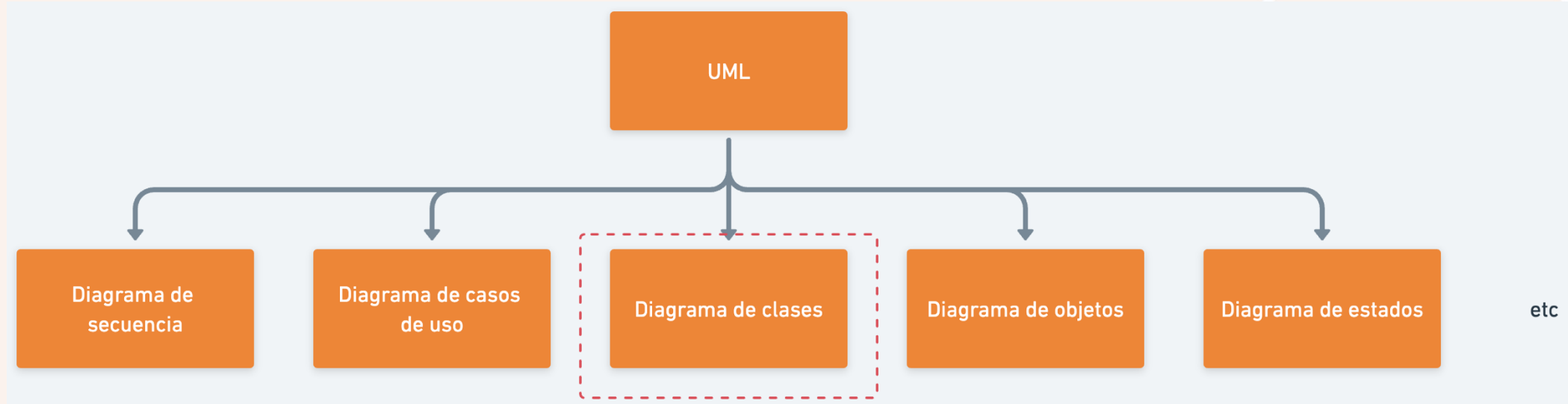
Identidad

Como se diferencia un objeto del
resto

Como diseñamos en POO?

Cuando hablamos de comenzar a desarrollar software escalable y altamente mantenible no es suficiente con conocer el lenguaje de programación, es necesario entender el problema, analizarlo y pensar de manera analítica una solución acorde al problema.

Lenguaje de Modelado Unificado

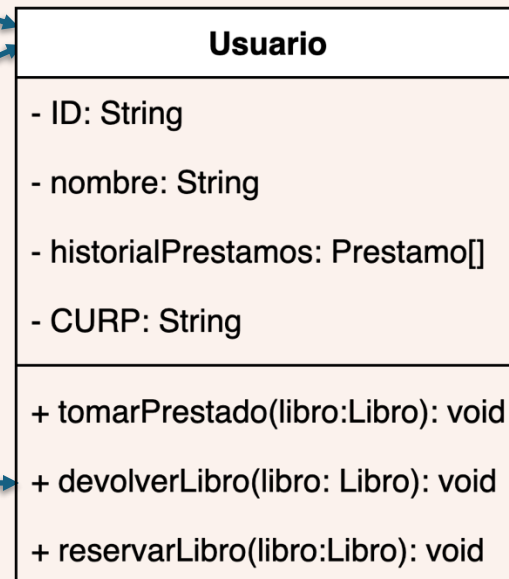


UML es un hechizo sencillo, pero inquebrantable

Sustantivo

Identificar clases y objetos.

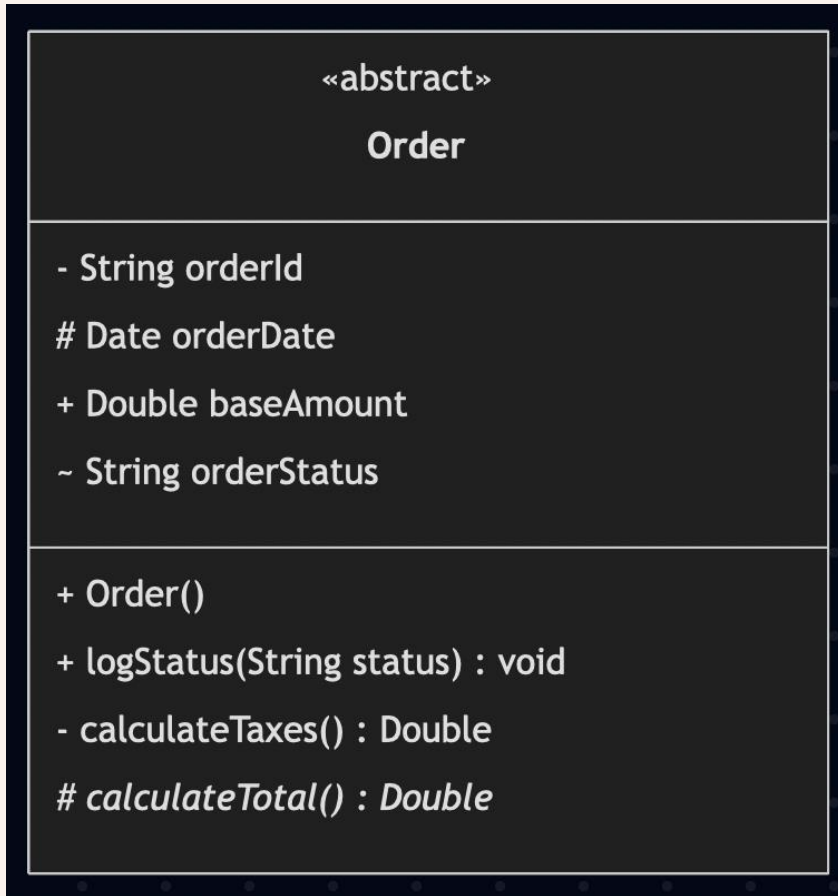
- Nombre
- Atributos
- Metodos



Verbo

Identificar las acciones de esas clases y objetos.

UML es un hechizo sencillo, pero inquebrantable



```
public abstract class Order {
    private String orderId;
    protected Date orderDate;
    public Double baseAmount;
    private String orderStatus;

    public Order() {}

    public void logStatus(String status) {
        // ...
    }

    private Double calculateTaxes(){
        return baseAmount*0.16;
    }

    protected abstract Double calculateTotal();
}
```


Modificadores de acceso



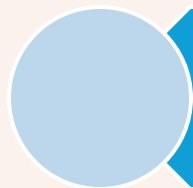
Publico (+)



Privado (-)



Protegido (#)



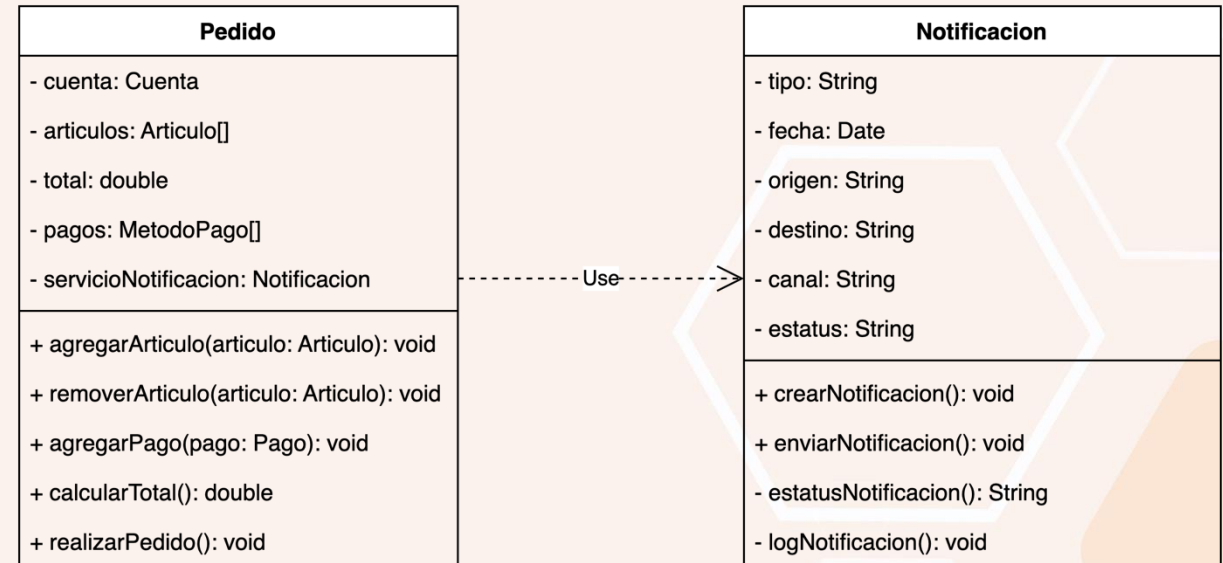
Default (~)

Tipos de relaciones entre clases

Dependencia(“utiliza un”)



Una clase depende de la operacion que ofrece otra clase



Al momento de realizar el pedido, se tiene que notificar al usuario que se ha realizado correctamente.

Tipos de relaciones entre clases

```
public class Notificacion {
    private String tipo;
    private Date fecha;
    private String origen;
    private String destino;
    private String canal;
    private String estatus;

    public Notificacion(String origen, String destino){
        ....
    }

    public void enviarNotificacion() throws Exception {
        this.estatus="PROCESANDO";
        this.logNotificacion();
        Thread.sleep(2000);
        this.estatus = "ENVIADO";
        this.fecha = new Date();
        this.logNotificacion();
        System.out.println("Notificación enviada a: " + this.destino + " por el canal: " + this.canal);
    }

    private String estatusNotificacion() {
        ...
    }

    private void logNotificacion() {
    }
}
```

```
public class Pedido {
    private Cuenta cuenta;
    private List<Articulo> articulos;
    private double total;
    private List<MetodoPago> pagos;
    private Notificacion servicioNotificacion;

    public Pedido(Cuenta cuenta) {
        ...
        servicioNotificacion = new Notificacion("PEDIDOSYA.COM", cuenta.obtenerCliente().obtenerCelular());
    }

    public void agregarArticulo(Articulo articulo) {
        ...
    }

    public void removerArticulo(Articulo articulo) {
        ...
    }

    public void agregarPago(MetodoPago pago) {
        ...
    }

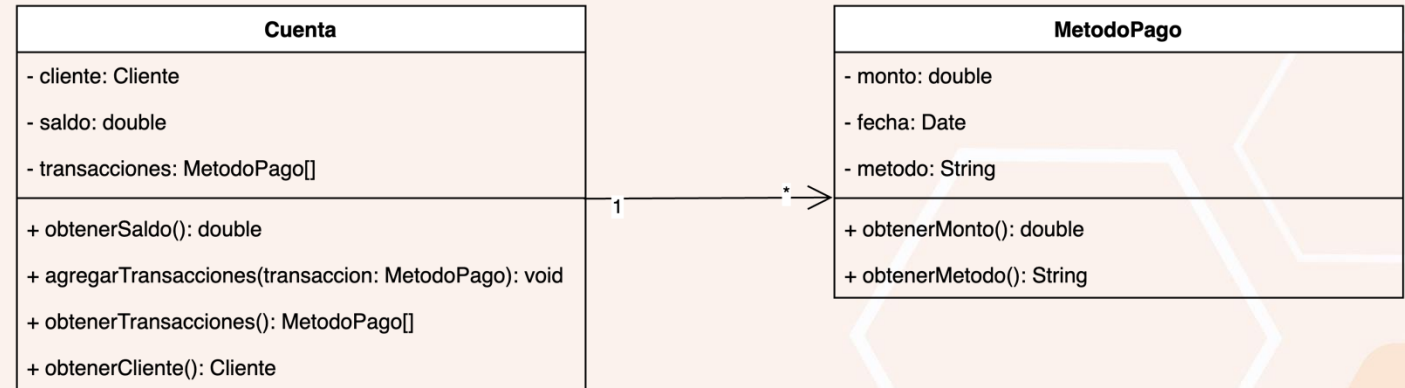
    public double calcularTotal() {
        ...
    }

    public void realizarPedido() throws Exception {
        servicioNotificacion.enviarNotificacion();
    }
}
```

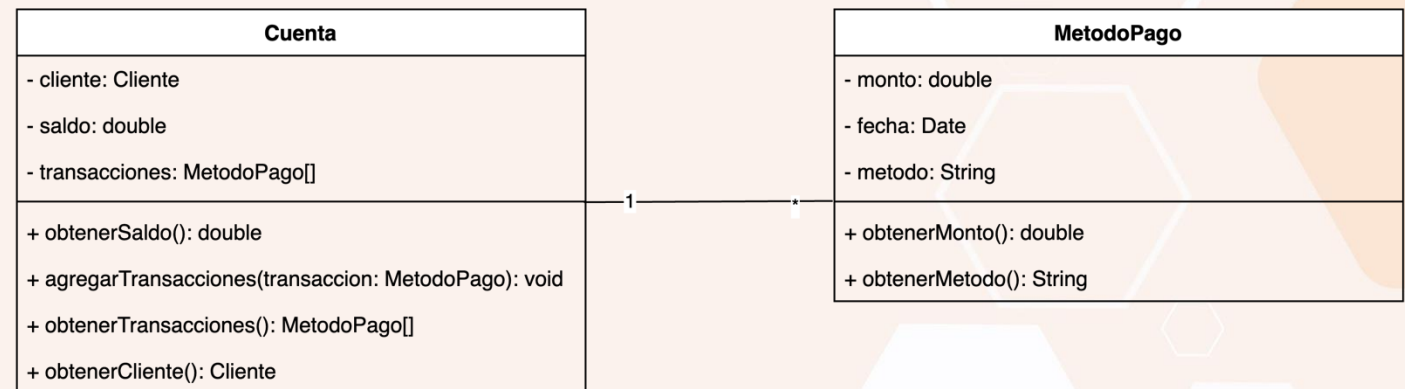
Tipos de relaciones entre clases

Una cuenta tendrá alguna transacción para poder realizar sus operaciones.

Asociación(“algún”)



Relacion entre dos unidades independientes.



Tipos de relaciones entre clases

Asociación(“algun”)



```
public class MetodoPago {
    private double monto;
    private Date fecha;
    private String metodo;

    public MetodoPago(double monto, String metodo) {
        this.monto = monto;
        this.fecha = new Date();
        this.metodo = metodo;
    }

    public double obtenerMonto() {
        return monto;
    }

    public String obtenerMetodo() {
        return metodo;
    }
}

public class Cuenta {
    private Cliente cliente;
    private double saldo;
    private List<MetodoPago> transacciones;

    public Cuenta(Cliente cliente){
        ...
    }

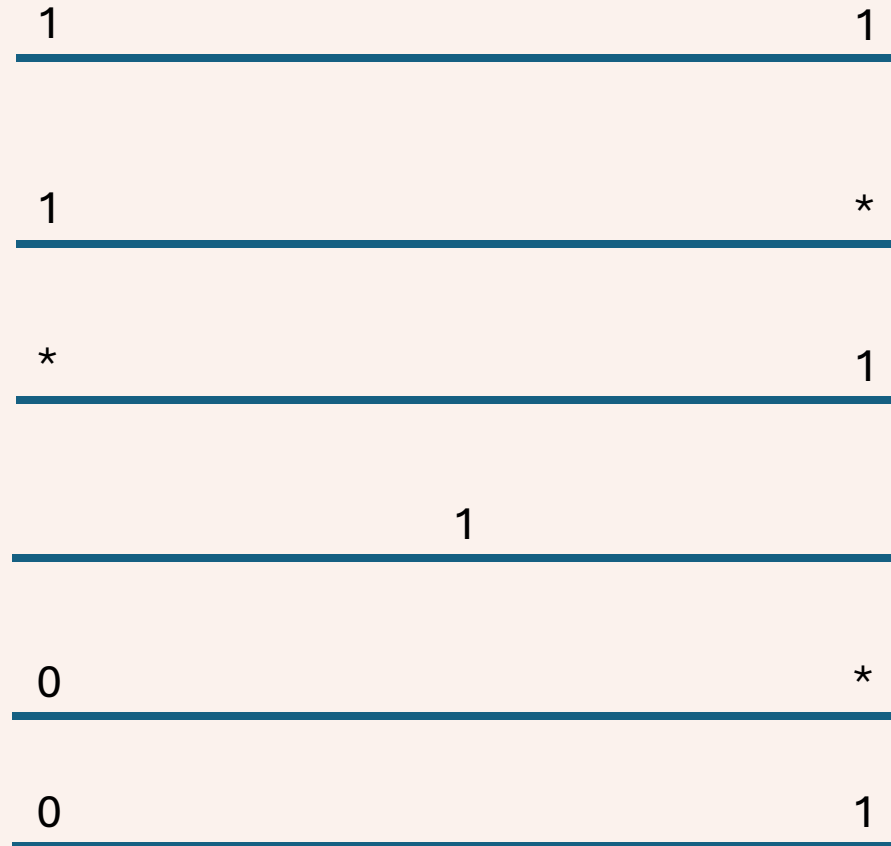
    public double obtenerSaldo() {
        ...
    }

    public void agregarTransaccion(MetodoPago transaccion) {
        transacciones.add(transaccion);
        saldo += transaccion.obtenerMonto();
    }

    public List<MetodoPago> obtenerTransacciones() {
        return transacciones;
    }

    public Cliente obtenerCliente() {
        ...
    }
}
```


Multiplicidad



Uno a Uno

Uno a Varios

Varios a Uno

Uno y solo uno

0 o Varios

0 o Uno

Tipos de relaciones entre clases

Asociación

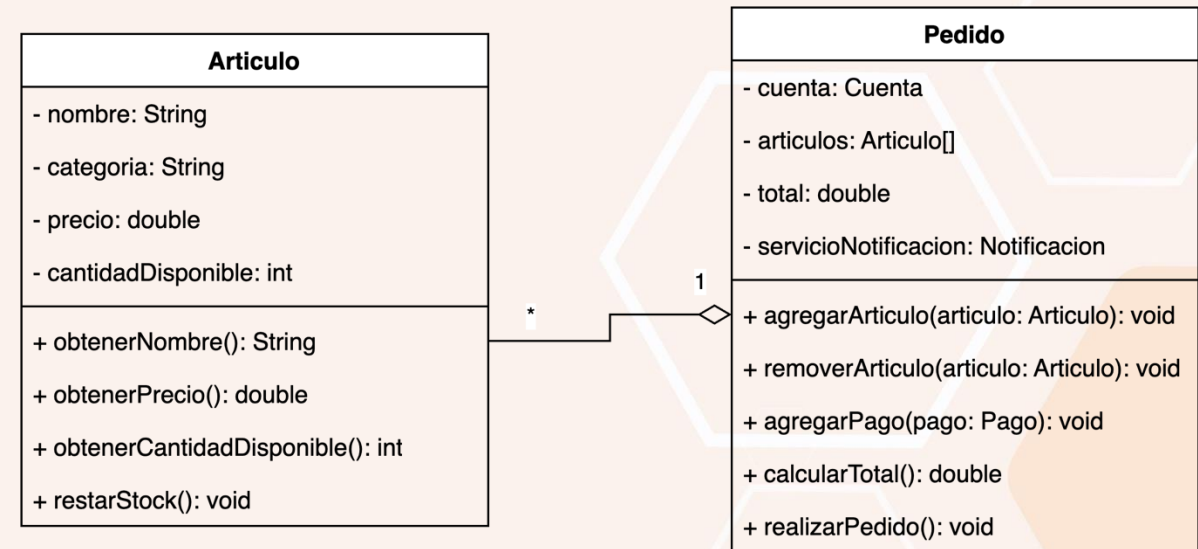
Un Pedido esta compuesto por multiples Articulos, pero los Articulos pueden existir independientemente del Pedido.

Agregación(“tiene un”)



Asociacion independiente entre dos objetos

Si la relacion se rompe, ambos siguen funcionando independientemente.



Tipos de relaciones entre clases

Asociación

Agregación(“tiene un”)



```
public class Articulo {
    private String nombre;
    private String categoria;
    private double precio;
    private int cantidadDisponible;

    public Articulo(String nombre, String categoria, double precio) {
        this.nombre = nombre;
        this.categoria = categoria;
        this.precio = precio;
        this.cantidadDisponible = 100;
    }

    public String obtenerNombre() {
        return nombre;
    }

    public double obtenerPrecio() {
        return precio;
    }

    public int obtenerCantidadDisponible() {
        return cantidadDisponible;
    }

    public void restarStock() {
        cantidadDisponible--;
    }
}

public class Pedido {
    private Cuenta cuenta;
    private List<Articulo> articulos;
    private double total;
    private Notificacion servicioNotificacion;

    public Pedido(Cuenta cuenta) {
        articulos = new ArrayList<Articulo>();
        this.cuenta = cuenta;
        servicioNotificacion = new Notificacion("PEDIDOSYA.COM", cuenta.obtenerCliente().obtenerCelular());
    }

    public void agregarArticulo(Articulo articulo) {
        articulos.add(articulo);
    }

    public void removerArticulo(Articulo articulo) {
        articulos.remove(articulo);
    }

    public void agregarPago(MetodoPago pago) {
        ...
    }

    public double calcularTotal() {
        return articulos.stream().reduce(0.0, (subtotal, articulo) -> subtotal + articulo.obtenerPrecio(), Double::sum);
    }

    public void realizarPedido() throws Exception {
        ...
    }
}
```

Tipos de relaciones entre clases

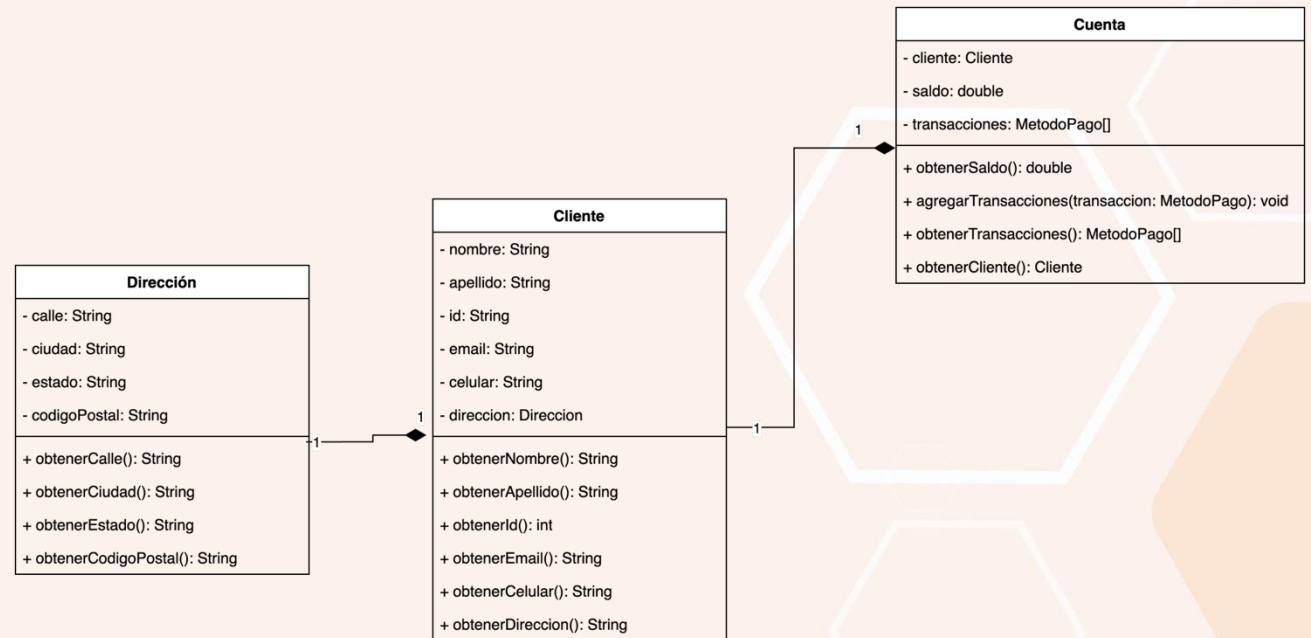
Asociación

Composicion(“debe tener un”)



Los objetos asociados no pueden existir independientemente del todo.

Asociacion Fuerte



Tipos de relaciones entre clases

Asociación

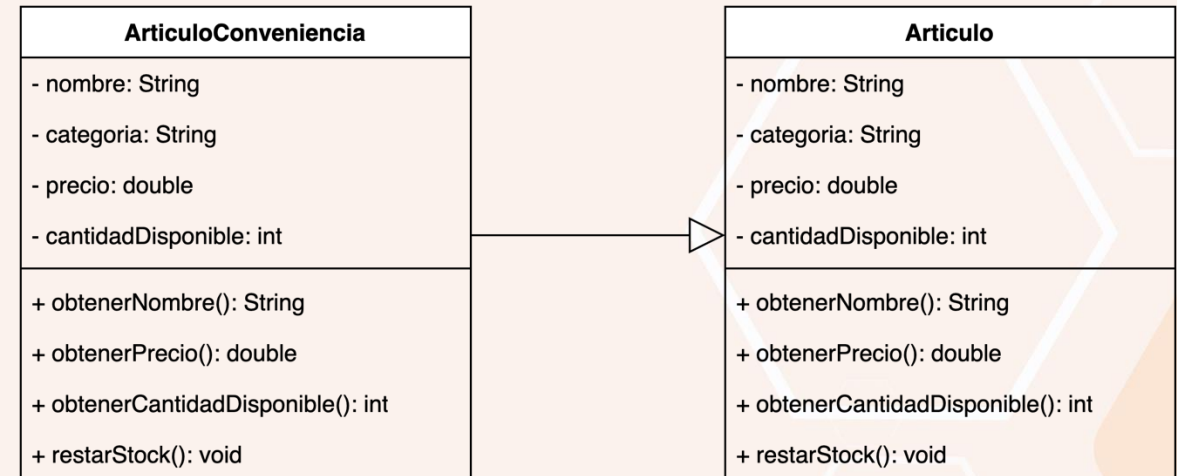
Composicion(“se compone de”)



```
public class Cliente {  
    private String nombre;  
    private String apellido;  
    private String id;  
    private String email;  
    private String celular;  
    private Direccion direccion;  
  
    public Cliente(String nombre, String apellido, String id, String email, String celular, Direccion direccion) {  
        this.nombre = nombre;  
        this.apellido = apellido;  
        this.id = id;  
        this.email = email;  
        this.celular = celular;  
        this.direccion = direccion;  
    }  
  
    public String obtenerNombre() {  
        return nombre;  
    }  
  
    public String obtenerApellido() {  
        return apellido;  
    }  
  
    public String obtenerId() {  
        return id;  
    }  
  
    public String obtenerEmail() {  
        return email;  
    }  
  
    public String obtenerCelular() {  
        return celular;  
    }  
  
    public Direccion obtenerDireccion() {  
        return direccion;  
    }  
}  
  
public class Direccion {  
    private String calle;  
    private String ciudad;  
    private String estado;  
    private String codigoPostal;  
  
    public String obtenerCalle() {  
        return calle;  
    }  
  
    public String obtenerCiudad() {  
        return ciudad;  
    }  
  
    public String obtenerEstado() {  
        return estado;  
    }  
  
    public String obtenerCodigoPostal() {  
        return codigoPostal;  
    }  
}
```

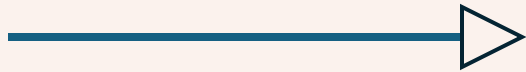
Tipos de relaciones entre clases

Herencia/Generalización(“Es un”)



Tipos de relaciones entre clases

Herencia/Generalización(“Es un”)



```
public class Artículo {  
    private String nombre;  
    private String categoria;  
    private double precio;  
    private int cantidadDisponible;  
  
    public Artículo(String nombre, String categoria, double precio) {  
        this.nombre = nombre;  
        this.categoria = categoria;  
        this.precio = precio;  
        this.cantidadDisponible = 100;  
    }  
  
    ....  
}  
  
public class ArtículoConveniencia extends Artículo {  
  
    public ArtículoConveniencia(String nombre, String categoria, double precio) {  
        super(nombre, categoria, precio);  
    }  
  
    @Override  
    public int obtenerCantidadDisponible() {  
        return super.obtenerCantidadDisponible();  
    }  
  
    @Override  
    public String obtenerNombre() {  
        return super.obtenerNombre();  
    }  
  
    @Override  
    public double obtenerPrecio() {  
        return super.obtenerPrecio();  
    }  
}
```

Tipos de relaciones entre clases

Realización(implementa)



Tipos de relaciones entre clases

Realización(implementa)



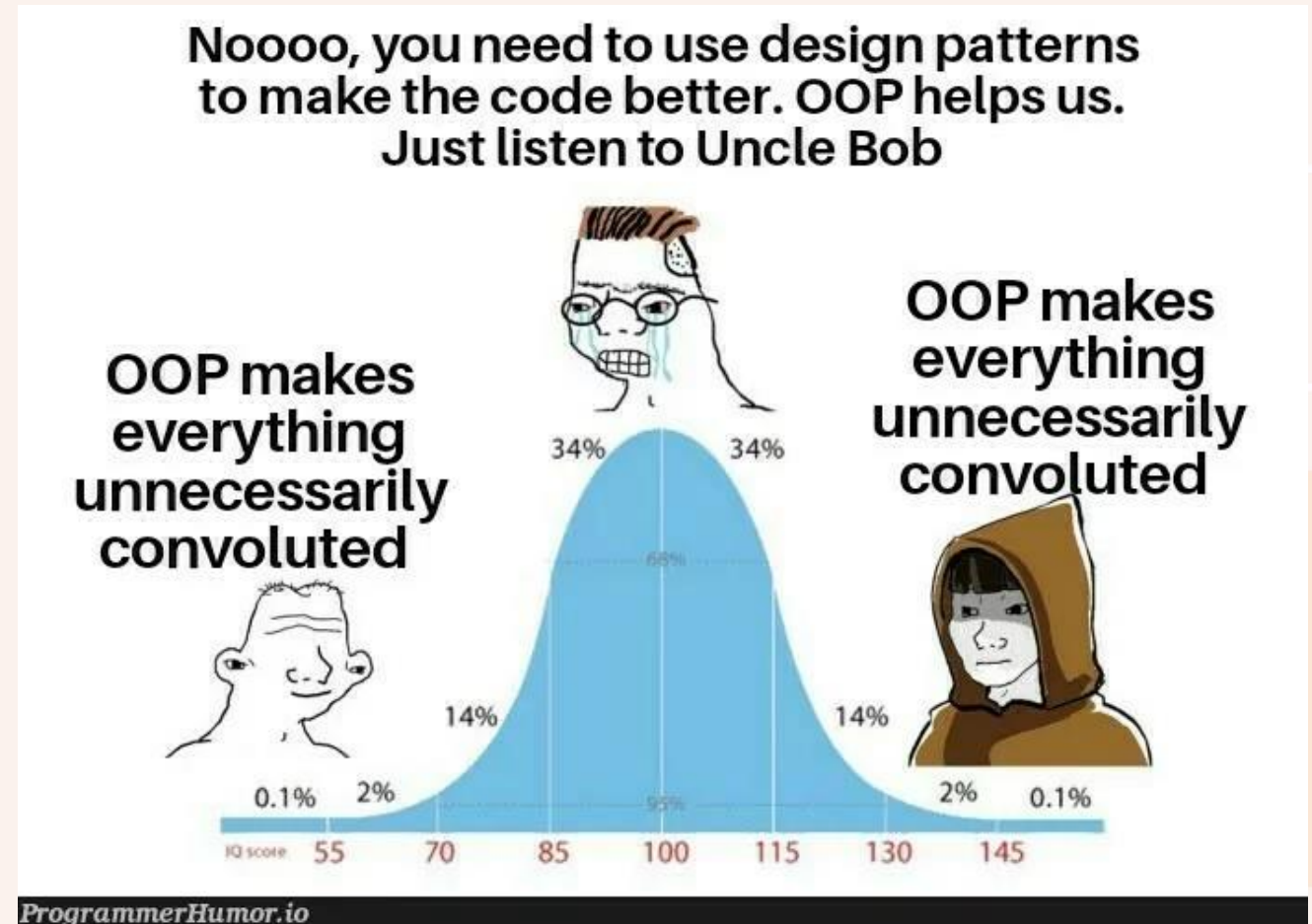
```
public class ProcesadorPedidos implements Procesador {  
  
    @Override  
    public void procesar(Pedido pedido) throws Exception {  
        pedido.realizarPedido();  
    }  
  
}  
  
public interface Procesador {  
  
    public void procesar(Pedido pedido) throws Exception;  
}
```

¿Por donde comenzar a diseñar?

- Diseño solido y bien estructurado.
 - Mantenibilidad
 - Reutilizacion
 - Extensibilidad
 - Testabilidad
 - Legibilidad y comprensión
- Identificación de requisitos y objetivos del sistema a diseñar.
 - Comunicación con stakeholders.
 - Recopilación de requisitos
 - Validación de requisitos
 - Objetivos

¿Para que diseñar?

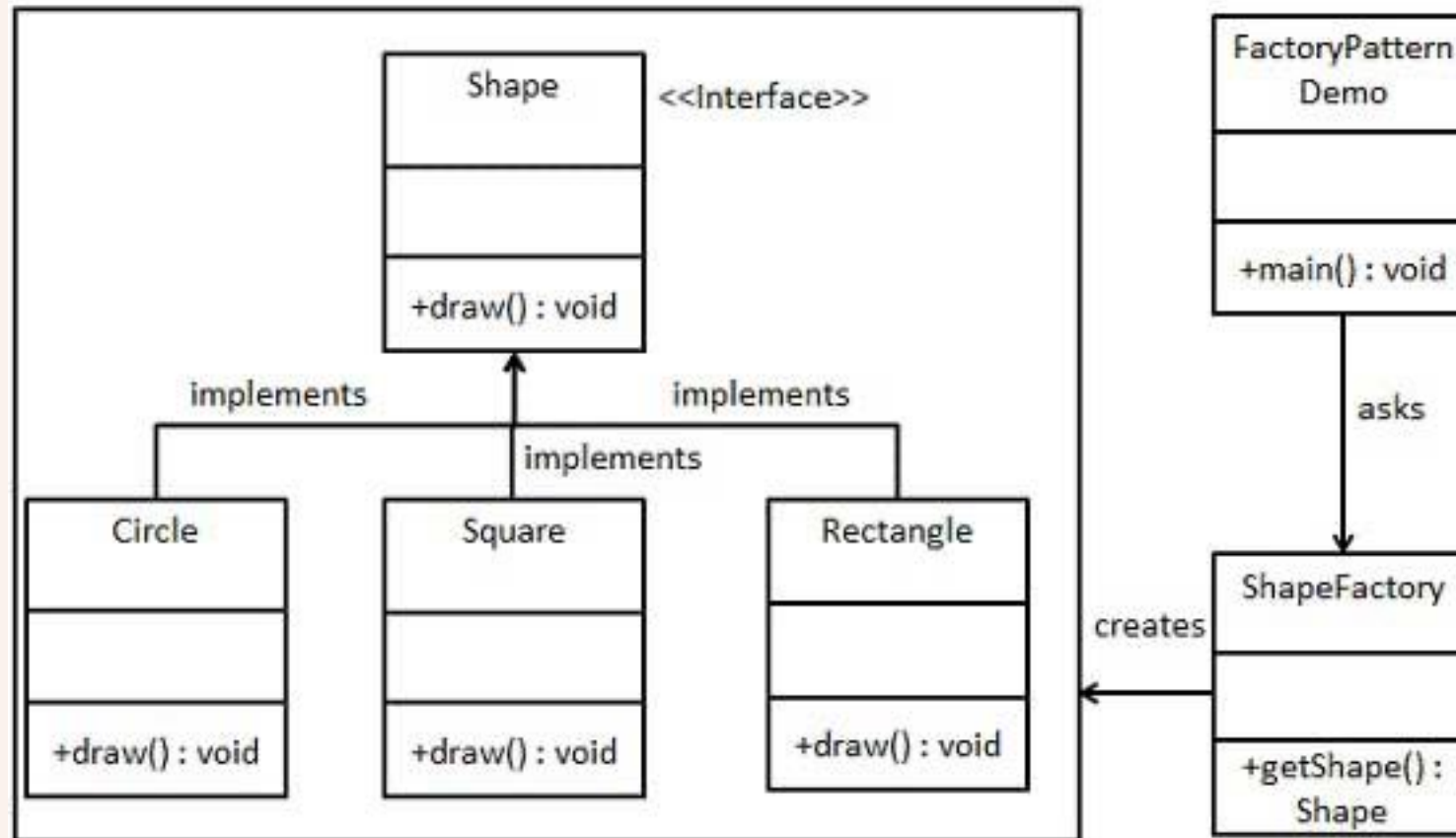
- Facilidad de mantenimiento
- Reutilización
- Modularidad
- Extensibilidad.



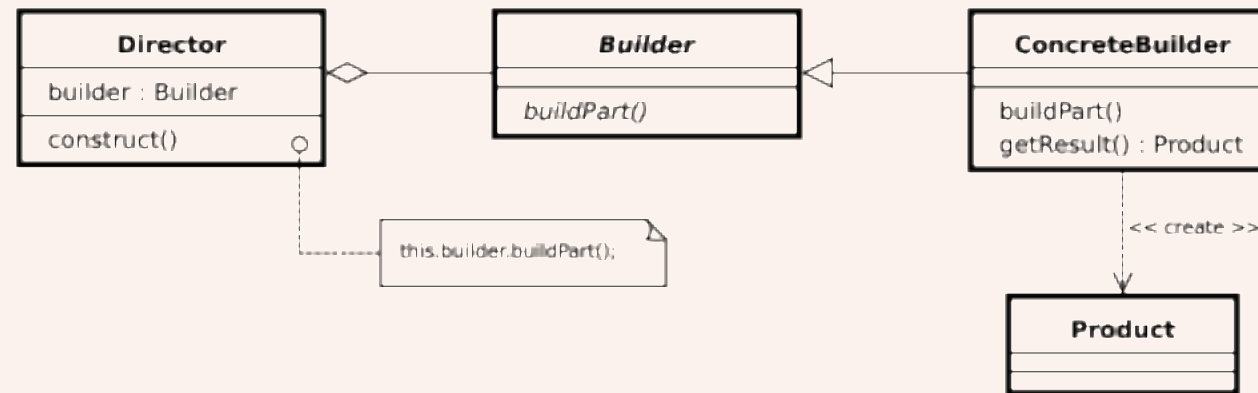
¿Quien va a diseñar?

- Arquitecto de software
- Analista de negocio
- Ingeniero de software
 - Diseña clases
 - Diseña patrones
 - Diseña Interfaces
- Tester
 - Diseña de pruebas

Ejemplo Diseño - Factory Pattern



Ejemplo Diseño - Builder Pattern



UML

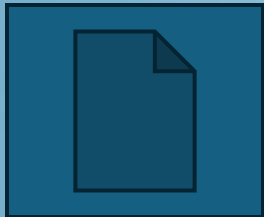
Los diagramas de clases UML son una herramienta poderosa para visualizar y comunicar la estructura y relaciones entre las clases en un sistema orientado a objetos.

- <https://www.visual-paradigm.com/solution/freeumltool/>
- <https://staruml.io/>
- <http://dia-installer.de/>
- <https://app.diagrams.net/>

Recomendado

- <https://plantuml.com/es/>
- <https://mermaid.js.org/>

Practica 1.2



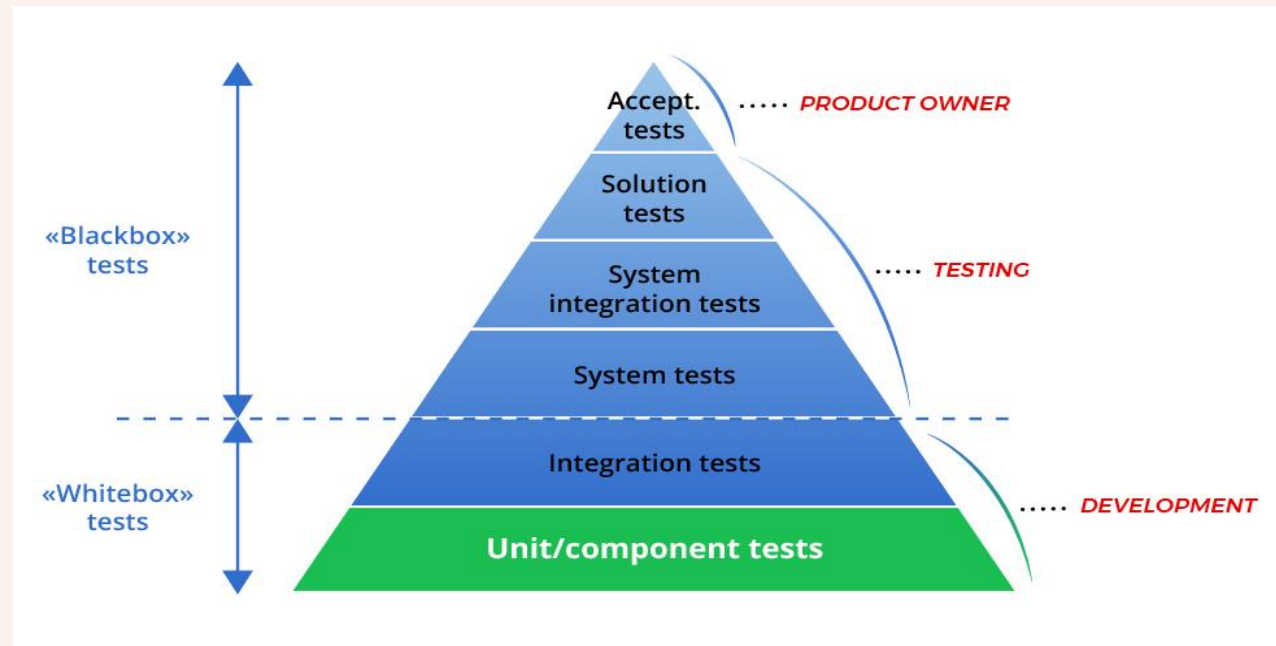


Pruebas unitarias(Unit Testing)

Comprobación del funcionamiento de **unidades de menor tamaño**.

Test que validan el comportamiento de un objeto y la lógica.

Se realiza mediante la fase de desarrollo.



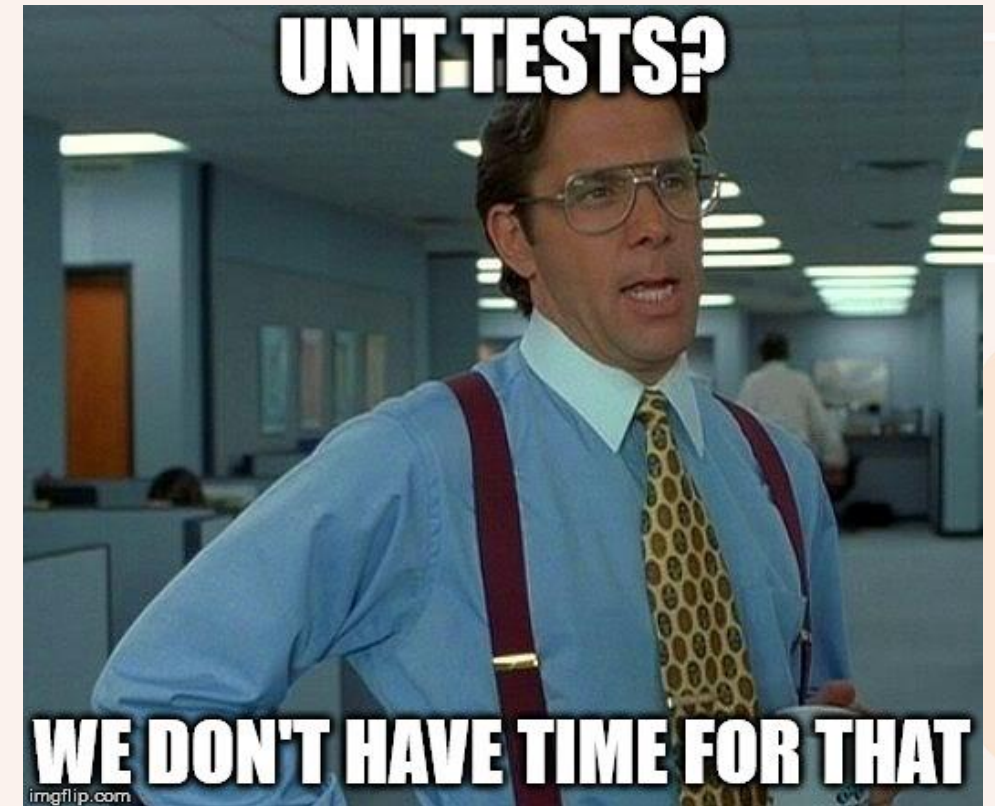
Beneficios



- Demuestran que la logica de negocio se encuentra en un buen estado y funcionara en todos los casos de uso.
- Aumentan legibilidad del código.
- Sirven como documentación del proyecto.
- Son muy rápidas de ejecutar.
- Da fiabilidad del código mediante un estándar de medición llamado *Coverage*.

¿Pruebas Unitarias == “Innecesarias”?

- Ahorro de tiempo
- Ahorro de dinero



¿Que entendemos por “unidad de menor tamaño”?

- Caso de uso
- Lógica de negocio
- Método
- Función
- Operación

```
1
2 class Pedido{
3     private Artículo[] articulos;
4     private double total;
5     private Pago[] pagos;
6     private Direccion direccionEntrega;
7     private Notificacion servicioNotificacion;
8
9     public Pedido(){
10         servicioNotificacion=new Notificacion();
11     }
12
13     public void agregarArticulo(Articulo articulo){
14         //ToDo
15         articulos[articulos.length+1]=articulo;
16     }
17
18     public void removerArticulo(Articulo articulo){
19         //ToDo
20     }
21
22     public void agregarPago(Pago pago){
23         //ToDo
24     }
25
26     public double calcularTotal(){
27         //ToDo
28     }
29
30     public void generarPedido(){
31         //ToDo
32     }
33
34 }
```

Ejemplos de pruebas para el método agregarArticulo

En caso de que el listado de los artículos se encuentre vacío, el articulo será el primer elemento

En caso de que el listado de los articulos se encuentre lleno, no se podra agregar un articulo mas.

En caso de que el listado de los articulos se encuentre lleno y no se puedan agregar mas articulos, recibir excepcion
ListadoArticulosLlenoException

¿Que entendemos por “unidad de menor tamaño”?

```
1
2 class Pedido{
3     private Artículo[] articulos;
4     private double total;
5     private Pago[] pagos;
6     private Direccion direccionEntrega;
7     private Notificacion servicioNotificacion;
8
9     public Pedido(){
10         servicioNotificacion=new Notificacion();
11     }
12
13     public void agregarArticulo(Articulo articulo){
14         //ToDo
15         articulos[articulos.length+1]=articulo;
16     }
17
18     public void removerArticulo(Articulo articulo){
19         //ToDo
20     }
21
22     public void agregarPago(Pago pago){
23         //ToDo
24     }
25
26     public double calcularTotal(){
27         //ToDo
28     }
29
30     public void generarPedido(){
31         //ToDo
32     }
33
34 }
```

Identificación de pruebas por criterios de aceptación Given, When, Then

Dado un listado de artículos

cuando se llame el método “agregarArticulo”

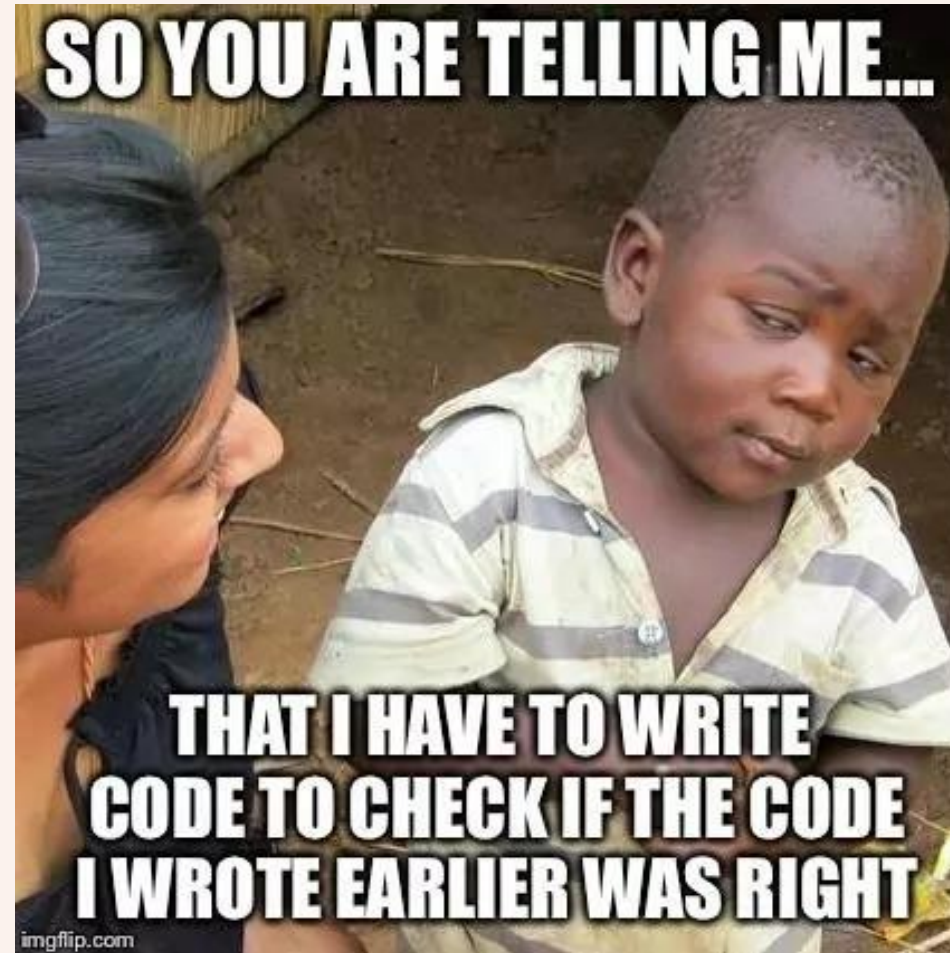
y reciba un parámetro de tipo articulo no nulo

y el listado se encuentre vacío

entonces se agregara el artículo como el primer elemento de la lista.

```
1 @Test
2 public void cuandoParametroNoSeaNuloYListadoVacioEntoncesAgregarEnLista(){
3     //...
4 }
5
6 @Test
7 public void whenParamNotNullAndListEmptyThenAddInList(){
8     //...
9 }
```

Pruebas unitarias(Unit Testing)



Buenas practicas para diseñar pruebas unitarias.

- Enfoque de cobertura
- Piensa en casos extremos
- Piensa en romper la unidad
- Nombres descriptivos y claros
- Pruebas simples y legibles
- Pruebas de regresión
- No llamar APIS, Conexiones de base de datos, peticiones de red, event hubs mientras se escriben testing.



Pruebas unitarias en JAVA

En java podemos utilizar la biblioteca “JUnit” para poder escribir pruebas unitarias completas. Sin embargo existen otras bibliotecas que nos ayudan a mejorar el diseño e implementación de estas pruebas.

- Mockito
- JMockit
- AssertJ
- TestNG



Conceptos clave JUnit

- **Aserciones(Assert)**
 - Equals
 - True
 - Null
 - NotNull
 - Exception
 - **Asunciones(Assumptions)**
 - **Mocks**
 - **Dummy**
- *Siempre se cumple el test.*
 - *Se asume algo para realizar un test.*
 - *Objeto que simula el estado y comportamiento de otro objeto.*
 - *Datos y objetos de prueba.*

Conceptos clave JUnit - Asserts

1. **assertEquals(expected, actual):** Compara si dos valores son iguales. Se utiliza para comparar tipos primitivos (como int, double, etc.) y objetos. Verifica si expected es igual a actual.
2. **assertTrue(condition):** Verifica si una condición es verdadera. Se utiliza para verificar que una expresión o condición booleana sea true.
3. **assertFalse(condition):** Verifica si una condición es falsa. Se utiliza para verificar que una expresión o condición booleana sea false.
4. **assertNull(object):** Verifica si un objeto es nulo.
5. **assertNotNull(object):** Verifica si un objeto no es nulo.
6. **assertSame(expected, actual):** Compara si dos objetos son la misma instancia. Verifica si expected y actual son exactamente el mismo objeto.
7. **assertNotSame(expected, actual):** Compara si dos objetos no son la misma instancia. Verifica si expected y actual no son el mismo objeto.
8. **assertArrayEquals(expectedArray, actualArray):** Compara si dos arreglos son iguales. Verifica si expectedArray y actualArray tienen los mismos elementos en el mismo orden.

Conceptos clave JUnit - Assumptions

1. **assumeTrue(condition):** Se asume que la condición especificada es verdadera. Si la condición es falsa, la prueba se ignora y se marca como pasada.
2. **assumeFalse(condition):** Se asume que la condición especificada es falsa. Si la condición es verdadera, la prueba se ignora y se marca como pasada.
3. **assumeNotNull(object):** Se asume que el objeto especificado no es nulo. Si el objeto es nulo, la prueba se ignora y se marca como pasada.
4. **assumeNull(object):** Se asume que el objeto especificado es nulo. Si el objeto no es nulo, la prueba se ignora y se marca como pasada.
5. **assumeNoException(exception):** Se asume que no se produce una excepción del tipo especificado. Si se produce la excepción, la prueba se ignora y se marca como pasada.
6. **assumeThat(value, matcher):** Se asume que el valor especificado cumple con el matcher proporcionado. Si no se cumple, la prueba se ignora y se marca como pasada. Este tipo de asunción utiliza la biblioteca de aserciones Hamcrest.

Conceptos clave JUnit - Mocks

```
1 import org.junit.Test;
2 import org.mockito.Mockito;
3
4 public class PedidoTest {
5
6     @Test
7     public void testGuardarPedido() {
8         // Crear el mock de la clase ConexionDB
9         ConexionDB conexionDBMock = Mockito.mock(ConexionDB.class);
10
11         // Crear una instancia de Pedido con el mock de ConexionDB
12         Pedido pedido = new Pedido(conexionDBMock);
13
14         // Definir el comportamiento esperado del mock
15         Mockito.when(conexionDBMock.guardarPedido(pedido)).thenReturn(true);
16
17         // Llamar al método a probar
18         boolean resultado = pedido.guardarPedido();
19
20         // Verificar el comportamiento esperado
21         Mockito.verify(conexionDBMock).guardarPedido(pedido);
22         assertTrue(resultado);
23     }
24 }
25
```

Conceptos clave JUnit

```
1
2 public class PedidoTest {
3
4     private Pedido pedido;
5     private Artículo articulo;
6     private ConexionDB conexionDBMock;
7
8     @Before
9     public void setUp() {
10         pedido = new Pedido();
11         conexionDBMock = new ConexionDBMock(); // Supongamos que tienes una implementación de simulación de la base de datos
12     }
13
14     @After
15     public void tearDown() {
16         System.out.println("Pedido Test Run!");
17     }
18
19     @Test
20     public void cuandoAgregarArticuloParametroNoSeaNuloYListadoVacioEntoncesAgregarEnLista() {
21         // Given
22         articulo = new Artículo();
23
24         // When
25         pedido.agregarArticulo(articulo);
26         conexionDBMock.guardarArticulo(articulo);
27
28         // Then
29         assertNotNull(articulo);
30         assertEquals(1, pedido.getArticulos().size());
31         assertTrue(pedido.getArticulos().contains(articulo));
32     }
33
34     // Otros test cases...
35 }
```


Organizar Unit Test

Patrón Given-When-Then(GWT)

```
1
2
3 @Test
4 public void cuandoAgregarArticuloParametroNoSeaNuloYListadoVacioEntoncesAgregarEnLista() {
5     // Given
6     articulo = new Articulo();
7
8     // When
9     pedido.agregarArticulo(articulo);
10    conexionDBMock.guardarArticulo(articulo);
11
12    // Then
13    assertNotNull(articulo);
14    assertEquals(1, pedido.getArticulos().size());
15    assertTrue(pedido.getArticulos().contains(articulo));
16 }
```

 Codye

Patrón Arrange-Act-Assert(AAA)

```
1
2
3 @Test
4 public void cuandoAgregarArticuloParametroNoSeaNuloYListadoVacioEntoncesAgregarEnLista() {
5     // Arrange
6     articulo = new Articulo();
7
8     // Act
9     pedido.agregarArticulo(articulo);
10    conexionDBMock.guardarArticulo(articulo);
11
12    // Assert
13    assertNotNull(articulo);
14    assertEquals(1, pedido.getArticulos().size());
15    assertTrue(pedido.getArticulos().contains(articulo));
16 }
```

 Codye

Temas de interés

- **Cobertura de pruebas(Testing Coverage)**
- **Desarrollo Guiado Por Pruebas(Test-Driven Development TDD)**
- **Pruebas de rendimiento y carga(Performance and Stress Testing)**
- **CI/CD**

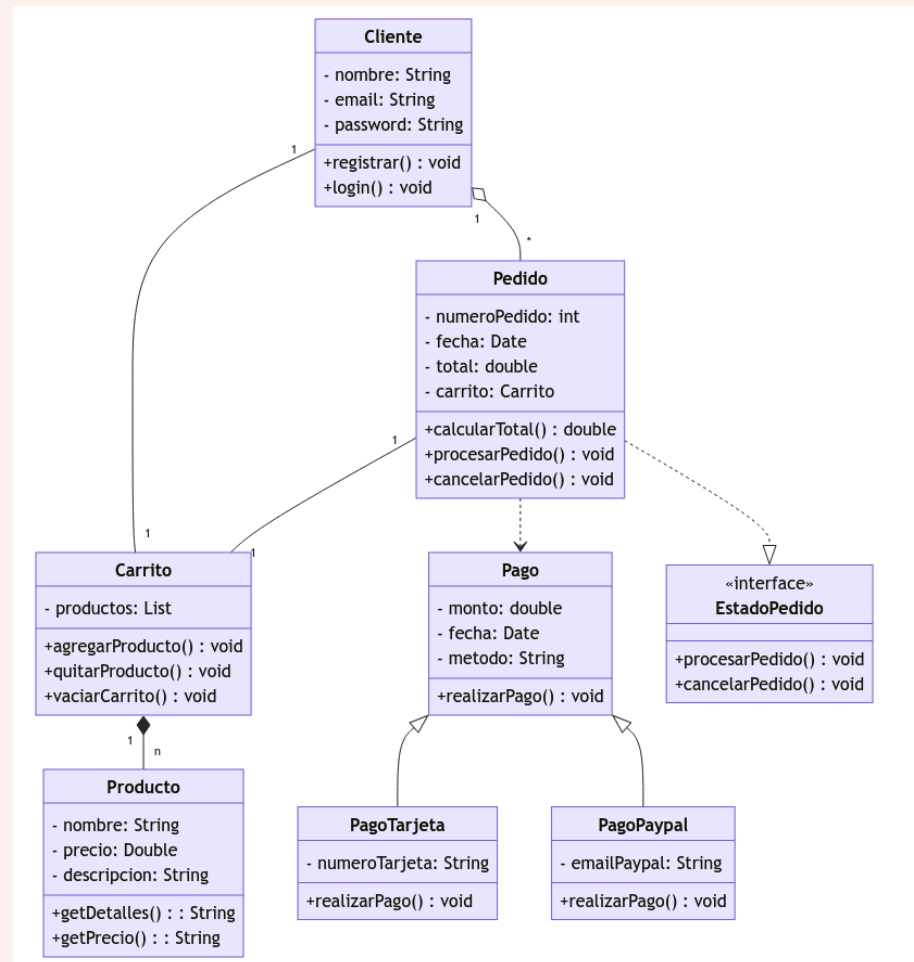


Practica 1.3



Práctica y ejercicios

- (Ejercicio) Escribir el código del diagrama UML de eCommerce.



Temas aprendidos

- Conceptos básicos de POO
- Vocabulario POO
- Relaciones entre clases
- UML
- Unit Testing

Contacto

Alfonso Hernández Xochipa

Ingeniero de Software | Arquitecto de Software

Hxa.dev@gmail.com

Tels: (+52) 2461847899

Redes sociales:



<https://www.linkedin.com/in/alfonso-hernandez-xochipa-a500ba190/>



<https://www.instagram.com/hernandez.xa/>



<https://github.com/hxadev>